

Workshop 4

Exception Handling

1. Write a Java program to demonstrate the difference between an Exception and an Error.

```
1 package week4;
2
3 public class Question1 {
4     public static void main(String[] args) {
5
6         System.out.println("Exception (can be handled):");
7
8         try {
9             int a = 10 / 0;
10        } catch (Exception e) {
11            System.out.println("Caught Exception: " + e);
12        }
13
14        System.out.println("Error (not really meant to be handled):");
15        try {
16            int size = Integer.MAX_VALUE;
17            int[] arr = new int[size];
18        } catch (Error err) {
19            System.out.println("Caught Error: " + err);
20        }
21    }
22 }
```

Console

```
<terminated> Question1 (3) [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.f
Exception (can be handled):
Caught Exception: java.lang.ArithmeticException: / by zero
Error (not really meant to be handled):
Caught Error: java.lang.OutOfMemoryError: Requested array size exceeds VM limit
```

2. Create a program that demonstrates ArithmeticException by dividing a number by zero and handle it using try-catch.

```
1 package week4;
2
3 class Question2 {
4     public static void main(String[] args) {
5         try {
6             int a = 10 / 0;
7         } catch (ArithmeticException e) {
8             System.out.println("Cannot divide by zero: " + e);
9         }
10    }
11 }
12
13
```

Console

```
<terminated> Question2 (2) [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hots
Cannot divide by zero: java.lang.ArithmeticException: / by zero
```

3. Write a Java program to show the complete Exception Hierarchy using comments and examples.

```
1 package week4;
2
3 /*
4  * JAVA EXCEPTION HIERARCHY
5  *
6  * Throwable
7  * 1.Exception(Checked Exceptions)
8  *   -IOException
9  *   -ClassNotFoundException
10  *   -Other checked exceptions
11  * 2.RuntimeException(Unchecked Exceptions)
12  *   -ArithmeticException
13  *   -NullPointerException
14  *   -ArrayIndexOutOfBoundsException
15  *   -Other runtime exceptions
16  * 3.Error
17  *   -StackOverflowError
18  *   -OutOfMemoryError
19  *   -Other JVM errors
20  */
21
22 import java.io.FileReader;
23 import java.io.IOException;
24
25 class Question3 {
26
27     public static void main(String[] args) {
28
29         //Checked Exception-IOException
30         try {
31             FileReader fr = new FileReader("data.txt");
32         } catch (IOException e) {
33             System.out.println("Caught exception: " + e);
34         }
35
36         //RuntimeException (Unchecked)
37         try {
38             int a = 10 / 0; // ArithmeticException
39         } catch (ArithmeticException e) {
40             System.out.println("Caught exception: " + e);
41         }
42
43         // Unchecked Exception-NullPointerException
44         try {
45             String str = null;
46             System.out.println(str.length()); // accessing method of null
47         } catch (NullPointerException e) {
48             System.out.println("Caught exception: " + e);
49         }
50
51         // Unchecked Exception-ArrayIndexOutOfBoundsException
52         try {
53             int[] arr = {1, 2, 3};
54             System.out.println(arr[5]); // index out of bounds
55         } catch (ArrayIndexOutOfBoundsException e) {
56             System.out.println("Caught exception: " + e);
57         }
58     }
59 }
60
61
```

Console X

```
<terminated> Question3 (2) [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.x86_64_21.0.
Caught exception: java.io.FileNotFoundException: data.txt (No such file or directory)
Caught exception: java.lang.ArithmeticException: / by zero
Caught exception: java.lang.NullPointerException: Cannot invoke "String.length()" because "str" is nul
Caught exception: java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 3
```

4. Develop a program to handle multiple types of exceptions (ArithmeticException, ArrayIndexOutOfBoundsException, and NullPointerException) in a single try block.

```
1 package week4;
2
3 class Question4 {
4     public static void main(String[] args) {
5         try {
6             int a = 10 / 0;
7             int arr[] = new int[2];
8             System.out.println(arr[5]);
9             String s = null;
10            System.out.println(s.length());
11        } catch (ArithmeticException e) {
12            System.out.println("Arithmetic Error");
13        } catch (ArrayIndexOutOfBoundsException e) {
14            System.out.println("Array index error");
15        } catch (NullPointerException e) {}
16        System.out.println("Null pointer error");
17    }
18 }
19
20
```

Console X

<terminated> Question4 (1) [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.j

Arithmetic Error

5. Write a program that demonstrates checked and unchecked exceptions with suitable examples.

```
1 package week4;
2
3 import java.io.FileReader;
4 import java.io.IOException;
5
6 class Question5 {
7
8     public static void main(String[] args) {
9
10        //Checked Exception-IOException
11        try {
12            FileReader fr = new FileReader("data.txt");
13        } catch (IOException e) {
14            System.out.println("Caught exception: " + e);
15        }
16
17        //RuntimeException (Unchecked)
18        try {
19            int a = 10 / 0; // ArithmeticException
20        } catch (ArithmeticException e) {
21            System.out.println("Caught exception: " + e);
22        }
23    }
24 }
25
```

Console X

<terminated> Question5 (1) [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx

Caught exception: [java.io.FileNotFoundException](#): data.txt (No such file or directory)

Caught exception: [java.lang.ArithmeticException](#): / by zero

6. Create a program that reads an integer from the user and throws a custom exception if the number is negative.

```
1 package week4;
2
3 import java.util.Scanner;
4
5 class NegativeNumberException extends Exception {
6     public NegativeNumberException(String msg) {
7         super("Number needs to be positive");
8     }
9 }
10
11 class Question6 {
12     public static void main(String[] args) throws NegativeNumberException {
13         Scanner sc = new Scanner(System.in);
14         System.out.println("Enter a num");
15         int num = sc.nextInt();
16
17         if (num < 0) {
18             throw new NegativeNumberException("");
19         }
20
21         System.out.println("Number is valid");
22     }
23 }
24 }
25
```

Console X

<terminated> Question6 (1) [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.x

Enter a num
-7
Exception in thread "main" week4.NegativeNumberException: Number needs to be positive
at week4.Question6.main(Question6.java:18)

7. Write a Java program using multiple catch blocks to handle different exceptions separately.

```
1 package week4;
2
3 class Question7 {
4     public static void main(String[] args) {
5         try {
6             int a = 10 / 2;
7             int arr[] = new int[2];
8             System.out.println(arr[5]);
9             String s = null;
10            System.out.println(s.length());
11        } catch (ArithmeticException e) {
12            System.out.println("Arithmetic Error");
13        } catch (ArrayIndexOutOfBoundsException e) {
14            System.out.println("Array index error");
15        } catch (NullPointerException e) {
16            System.out.println("Null pointer error");
17        }
18    }
19 }
20
```

Console X

<terminated> Question7 (2) [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse

Array index error

8. Create a program demonstrating nested try-catch blocks.

```
1 package week4;
2
3 class Question8 {
4     public static void main(String[] args) {
5         try {
6             try {
7                 int a = 5 / 0;
8             } catch (ArithmeticException e) {
9                 System.out.println("Inner catch");
10            }
11            int b = 9 / 0;
12        } catch (Exception e) {
13            System.out.println("Outer catch");
14        }
15    }
16 }
17
18
```

Console X

<terminated> Question8 (1) [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse

Inner catch
Outer catch

9. Write a Java program that uses the throw keyword to manually throw an exception when invalid input is entered.

```
1 package week4;
2
3 class Question9 {
4     public static void main(String[] args) {
5         int age = -5;
6         if (age < 0)
7             throw new RuntimeException("Invalid age");
8     }
9 }
10
11
```

Console X

<terminated> Question9 (2) [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk

Exception in thread "main" java.lang.RuntimeException: Invalid age
at week4.Question9.main(Question9.java:7)

10. Develop a Java program that uses the throws keyword in a method and handle the exception in the calling method.

```
1 package week4;
2
3 import java.io.*;
4
5 class Question10 {
6     static void readFile() throws IOException {
7         FileInputStream f = new FileInputStream("test.txt");
8     }
9
10    public static void main(String[] args) {
11        try {
12            readFile();
13        } catch (IOException e) {
14            System.out.println("Handled: " + e);
15        }
16    }
17 }
18
19
```

Console X

<terminated> Question10 (2) [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx...
Handled: java.io.FileNotFoundException: test.txt (No such file or directory)

11. Write a Java program to demonstrate the use of the finally block in exception handling.

```
1 package week4;
2
3 import java.io.*;
4
5 class Question11 {
6     public static void main(String[] args) throws IOException {
7         FileReader fr = null;
8         try {
9             fr = new FileReader("/Users/mallika/Desktop/java/data.txt");
10        } catch (FileNotFoundException e) {
11            System.out.println("File does not exist");
12        } finally {
13            if (fr != null) {
14                fr.close();
15                System.out.println("File closed successfully in finally block.");
16            }
17        }
18    }
19 }
20
21
```

Console X

<terminated> Question11 (2) [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx...
File closed successfully in finally block.

12. Create a program to show that the finally block executes even when return is used inside a try or catch block.

```
1 package week4;
2
3 class Question12 {
4     public static int test() {
5         try {
6             return 5;
7         } finally {
8             System.out.println("Finally");
9         }
10    }
11
12    public static void main(String[] args) {
13        test();
14    }
15 }
16
```

Console X

<terminated> Question12 (2) [Java Application] /Users/mallika/.p2/pool/plug
Finally

13. Write a program to demonstrate try-with-resources by reading data from a file safely.

```
1 package week4;
2
3 import java.io.*;
4
5 class Question13 {
6     public static void main(String[] args) {
7
8         try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
9             String line;
10            while ((line = br.readLine()) != null) {
11                System.out.println(line);
12            }
13        } catch (IOException e) {
14            System.out.println("Caught Exception: " + e);
15        }
16    }
17 }
18
19
```

Console X

<terminated> Question13 (1) [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.
Caught Exception: [java.io.FileNotFoundException](#): data.txt (No such file or directory)

14. Develop a Java program to handle multiple exceptions using multi-catch syntax (catch(Exception1 | Exception2 e)).

```
1 package week4;
2
3 class Question14 {
4     public static void main(String[] args) {
5         try {
6             int a = 10 / 0;
7             int arr[] = new int[2];
8             System.out.println(arr[5]);
9         } catch (ArithmeticException | ArrayIndexOutOfBoundsException e) {
10             System.out.println("Handled: " + e);
11         }
12     }
13 }
14
15
```

Console X

<terminated> Question14 (1) [Java Application] /Users/mallika.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.fu
Handled: [java.lang.ArithmeticException](#): / by zero

15. Write a program that shows how an exception is propagated from one method to another using throws.

```
1 package week4;
2
3 /* propagation is a process in which the exception is dropped from the top
4  to the bottom of the stack. If not caught once, the exception again drops
5  down to the previous method, and so on until it gets caught or until it
6  reaches the very bottom of the call stack.
7  */
8 class Question15 {
9
10     //method that throws an exception
11     static void methodA() throws Exception {
12         throw new Exception("Exception thrown in methodA");
13     }
14
15     //method that calls methodA and also declares throws
16     static void methodB() throws Exception {
17         methodA(); //exception propagates to methodB
18     }
19
20     public static void main(String[] args) {
21         try {
22             methodB(); //exception propagated to main
23         } catch (Exception e) {
24             System.out.println("Exception caught in main: " + e);
25         }
26     }
27 }
28
```

Console X

<terminated> Question15 [Java Application] /Users/mallika.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.
Exception caught in main: [java.lang.Exception](#): Exception thrown in methodA

16. Create a program to demonstrate the concept of rethrowing an exception.


```
1 package week4;
2
3 class Question16 {
4     public static void main(String[] args) {
5         try {
6             try {
7                 int a = 10 / 0;
8             } catch (ArithmeticException e) {
9                 System.out.println("exception caught, rethrowing...");
10                throw e;
11            }
12        } catch (ArithmeticException e) {
13            System.out.println("Caught in outer catch: " + e);
14        }
15    }
16 }
17
```

Console ×

<terminated> Question16 [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot

exception caught, rethrowing...

Caught in outer catch: [java.lang.ArithmeticException](#): / by zero

17. Write a Java program to demonstrate the difference between checked and unchecked exceptions by creating your own custom checked exception.

```
1 package week4;
2
3 //custom checked exception
4 class CheckedException extends Exception {
5     public CheckedException(String message) {
6         super(message);
7     }
8 }
9
10 class Question17 {
11
12     //checked exception method
13     static void checkedExample() throws CheckedException {
14         throw new CheckedException("custom checked exception");
15     }
16
17     //unchecked exception method
18     static void uncheckedExample() {
19         int a = 10 / 0;
20     }
21
22     public static void main(String[] args) {
23         //handling checked exception
24         try {
25             checkedExample();
26         } catch (CheckedException e) {
27             System.out.println("Checked Exception caught: " + e);
28         }
29
30         //handling unchecked exception
31         try {
32             uncheckedExample();
33         } catch (ArithmeticException e) {
34             System.out.println("Unchecked Exception caught: " + e);
35         }
36     }
37 }
38
```

Console X

<terminated> Question17 [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre

Checked Exception caught: week4.CheckedException: custom checked exception

Unchecked Exception caught: java.lang.ArithmeticException: / by zero

18. Create a class InvalidAgeException and write a program that throws this exception if a person's age is below 18.

```
1 package week4;
2
3 class AgeException extends RuntimeException{
4     public AgeException(String s) {
5         super("Age should be above 18");
6     }
7 }
8
9 public class Question18 {
10     public static void main(String[] args) throws Exception{
11
12         int age;
13
14         age = 17;
15
16         if(age<18) {
17             throw new AgeException("");
18         }
19         System.out.println(age);
20     }
21 }
22
```

Console X

<terminated> Question18 [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot

Exception in thread "main" week4.AgeException: Age should be above 18
at week4.Question18.main(Question18.java:17)

19. Write a Java program to handle user input errors gracefully using exception handling.

```
1 package week4;
2
3 import java.util.Scanner;
4
5 class Question19 {
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8
9         try {
10             System.out.print("Enter an integer: ");
11             int num = sc.nextInt();
12             System.out.println("You entered: " + num);
13         } catch (Exception e) {
14             System.out.println("enter valid integer");
15         } finally {
16             sc.close();
17         }
18     }
19 }
20
21
```

Console X

<terminated> Question19 [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.just

Enter an integer: ab
enter valid integer

20. Develop a Java program to demonstrate exception chaining (new Exception("msg", cause)).

```
1 package week4;
2
3 class Question20 {
4
5     // method causing original exception
6     static void methodA() throws Exception {
7         try {
8             int result = 10 / 0; // causes ArithmeticException
9         } catch (ArithmeticException e) {
10             // wrapping the original exception into a new exception(chaining)
11             throw new Exception("Exception in methodA due to division by zero", e);
12         }
13     }
14
15     public static void main(String[] args) {
16         try {
17             methodA();
18         } catch (Exception e) {
19             System.out.println("Caught Exception: " + e);
20         }
21     }
22 }
23
```

Console X

<terminated> Question20 [Java Application] /Users/mallika.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.x86_64
Caught Exception: java.lang.Exception: Exception in methodA due to division by zero

21. Create a program to demonstrate how finally is used for closing resources like files or database connections.

```
1 package week4;
2
3 import java.io.*;
4
5 class Question21 {
6     public static void main(String[] args) throws IOException {
7         FileReader fr = null;
8         try {
9             fr = new FileReader("/Users/mallika/Desktop/java/data.txt");
10        } catch (FileNotFoundException e) {
11            System.out.println("File does not exist");
12        } finally {
13            if (fr != null) {
14                fr.close();
15                System.out.println("File closed successfully in finally block.");
16            }
17        }
18    }
19 }
```

Console X

<terminated> Question21 [Java Application] /Users/mallika.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.
File closed successfully in finally block.

22. Write a program that takes a filename from the user, reads its content, and handles `FileNotFoundException`.

```
5 import java.util.Scanner;
6
7 class Question22 {
8     public static void main(String[] args) {
9         Scanner inputScanner = new Scanner(System.in);
10        System.out.print("Enter filename: ");
11        String filename = inputScanner.nextLine();
12
13        File file = new File(filename);
14
15        try {
16            Scanner fileScanner = new Scanner(file);
17            System.out.println("File content:");
18            while (fileScanner.hasNextLine()) {
19                System.out.println(fileScanner.nextLine());
20            }
21            fileScanner.close();
22        } catch (FileNotFoundException e) {
23            System.out.println("File not found: " + filename);
24        } finally {
25            inputScanner.close();
26        }
27    }
28 }
29
```

Console X

<terminated> Question22 [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.open
Enter filename: abc.txt
File not found: abc.txt

23. Write a program that demonstrates how to create and handle multiple custom exceptions in a single program.

```
1 package week4;
2
3 import java.util.Scanner;
4
5 class NameException extends RuntimeException {
6     public NameException() {
7         super("Name cannot be null or empty");
8     }
9 }
10
11 class AgeException1 extends RuntimeException {
12     public AgeException1() {
13         super("Age must be positive");
14     }
15 }
16
17 class Question23 {
18     public static void main(String[] args) {
19         Scanner sc = new Scanner(System.in);
20
21         try {
22             System.out.print("Enter your name: ");
23             String name = sc.nextLine();
24             if (name == null || name.isEmpty()) {
25                 throw new NameException();
26             }
27
28             System.out.print("Enter your age: ");
29             int age = sc.nextInt();
30             if (age < 0) {
31                 throw new AgeException1();
32             }
33
34             System.out.println("Name: " + name + ", Age: " + age);
35
36         } catch (NameException | AgeException1 e) {
37             System.out.println("Custom Exception: " + e);
38         } catch (Exception e) {
39             System.out.println("Other Exception: " + e);
40         } finally {
41             sc.close();
42         }
43     }
44 }
45
```

Console ×

<terminated> Question23 [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jr
Enter your name: gd
Enter your age: -2
Custom Exception: week4.AgeException1: Age must be positive

24. Create a Java program to demonstrate how exception handling improves program reliability and flow control.

```
1 package week4;
2
3 import java.util.Scanner;
4
5 class Question24 {
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8
9         try {
10             System.out.print("Enter a number to divide 100 by: ");
11             int num = sc.nextInt();
12             int result = 100 / num;
13             System.out.println("Result: " + result);
14         } catch (ArithmeticException e) {
15             System.out.println("Error: Cannot divide by zero.");
16         }
17
18         // Program continues after exception handling
19         System.out.println("Program continues to run...");
20         System.out.println("Example:");
21         int sum = 1+6;
22         System.out.println("Sum of 1 and 6: "+ sum);
23
24         sc.close();
25     }
26 }
27
```

Console X

<terminated> Question24 [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.h
Enter a number to divide 100 by: 0
Error: Cannot divide by zero.
Program continues to run...
Example:
Sum of 1 and 6: 7

25. Develop a Java program that demonstrates the difference between throw and throws with examples in different methods.

```
1 package week4;
2
3 class InvalidAgeException extends Exception {
4     public InvalidAgeException() {
5         super("Age cannot be less than 0");
6     }
7 }
8
9 public class Question25 {
10
11     //method that declares it may throw an exception (throws)
12     static void validateAge(int age) throws InvalidAgeException {
13         if (age < 0) {
14             //actually throws the exception
15             throw new InvalidAgeException();
16         }
17         System.out.println("Age is valid: " + age);
18     }
19
20     public static void main(String[] args) {
21         int age = -5;
22
23         try {
24             //call method that may throw exception
25             validateAge(age);
26         } catch (InvalidAgeException e) {
27             System.out.println("Exception caught in main: " + e);
28         }
29     }
30 }
31
32
33
```

Console X

<terminated> Question25 [Java Application] /Users/mallika/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.
Exception caught in main: week4.InvalidAgeException: Age cannot be less than 0